Data Analysis from TP

Multi-Agent Systems for Smart Machining - ECIU University

Chloé Fuchs, Paolo Rossi, Daniele Trisotto

2025-11-19

Introduction

The machining process under analysis consists of three main operations: surface milling, drilling, and pocket milling. This document compiles the analyses of the various machining operations and ends with a dedicated section that provides a comparison of the corresponding results.

By examining the power profile over time, it is possible to identify the different phases of the machining process. The Figure 2 shows the total power P_{tot} measured during the cutting session, together with the cutting power component P_c, computed as $P_c = P_{tot} - P_{nc}$, where P_{nc} represents the no-load power measured during the NON-cutting phase.

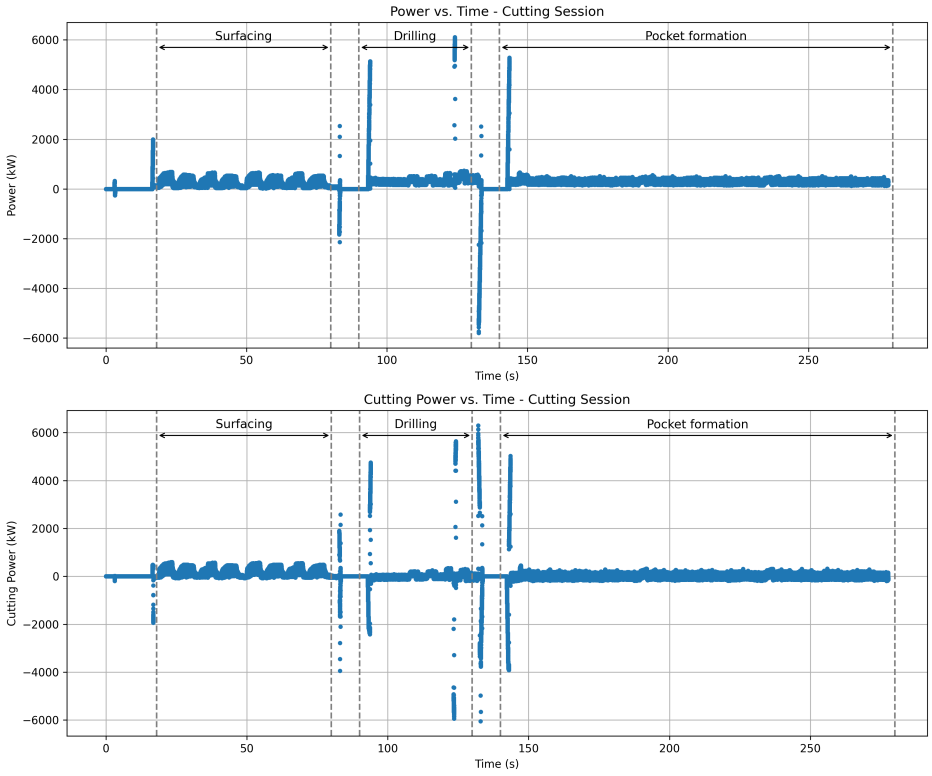


Figure 1: Total power P_{tot} and cutting power P_c during the surface milling operation.

Ignoring the instantaneous power peaks, it can be observed that the cutting power frequently becomes negative, which is physically impossible, as it would imply that the cutting process generates energy. This

behaviour suggests an inconsistency between the cutting and NON-cutting sessions. For this reason, the analysis will be carried out using the total power measured during the cutting session. This choice remains fully representative, since including the no-load power would simply result in a constant offset of the power values.

Each operation was analyzed by one ECIU student:

- Paolo Rossi: surface milling;
- Chloé Fuchs: drilling;
- Daniele Trisotto: pocket milling.

Surface Milling (Paolo)

Introduction

The machining process under analysis consists of three main operations: surface milling, drilling, and pocket milling. In the following section, the surface milling phase is examined in detail.

Based on the information provided by the students of INSA Toulouse and on the ShopMill code `E_MI_PL("FR_SURF_D63_ALU","FORET5.5_CARB_FSI",...);*RO*`, the following machining parameters are defined:

- Tool diameter: $D = 63\,\text{mm}$
- Cutting speed: $V_c = 150\,\text{m/min}$
- Spindle speed: $N = \frac{V_c}{\pi D} = 757.88$ rpm

By examining the power profile over time, it is possible to identify the different phases of the machining process. The Figure 2 shows the total power P_{tot} measured during the cutting session, together with the cutting power component P_c, computed as $P_c = P_{tot} - P_{nc}$, where P_{nc} represents the no-load power measured during the NON-cutting phase.

Ignoring the instantaneous power peaks, it can be observed that the cutting power frequently becomes negative, which is physically impossible, as it would imply that the cutting process generates energy. This behaviour suggests an inconsistency between the cutting and NON-cutting sessions. For this reason, the analysis will be carried out using the total power measured during the cutting session. This choice remains fully representative, since including the no-load power would simply result in a constant offset of the power values.

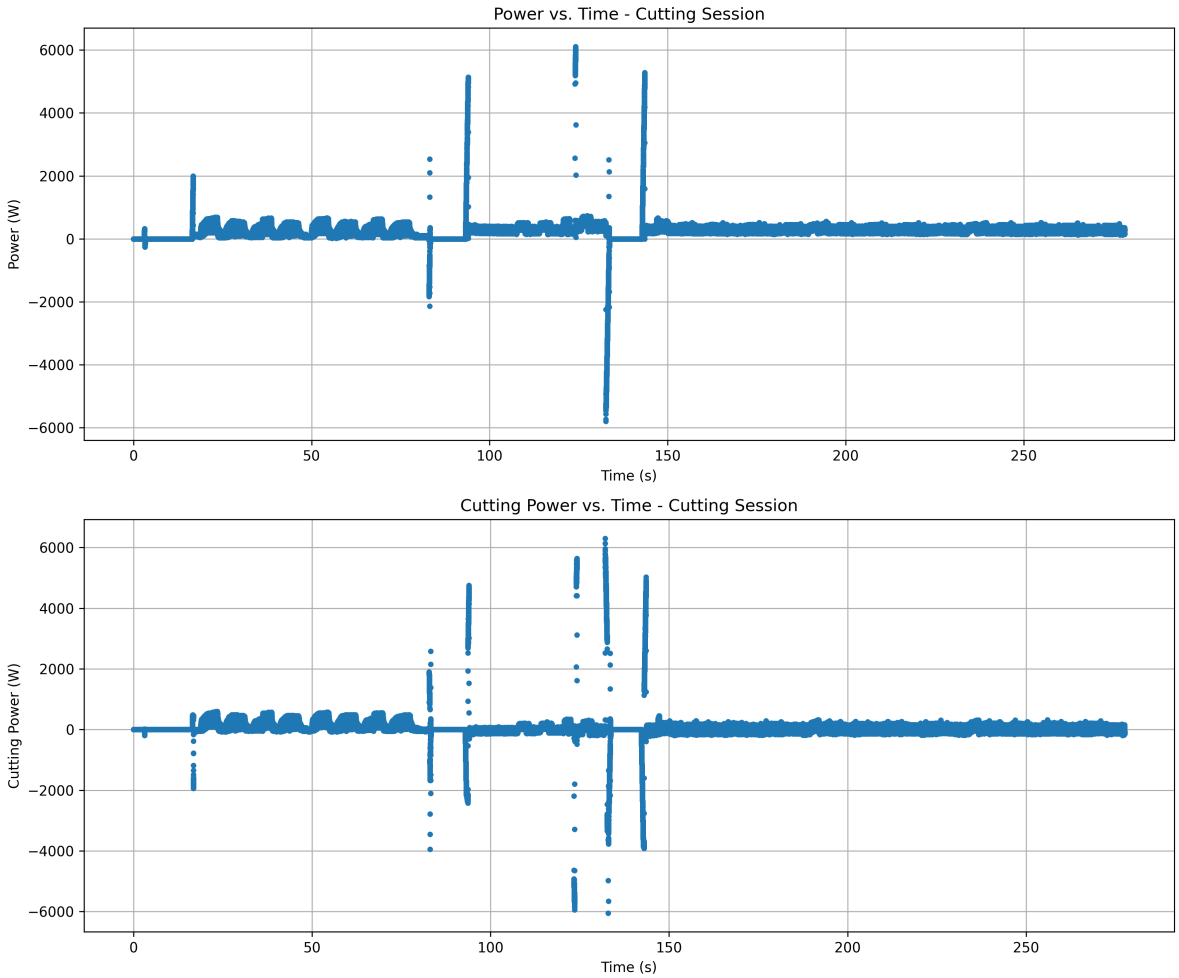


Figure 2: Total power P_{tot} and cutting power P_c during the surface milling operation.

Power Consumption per Revolution

Based on the power trend shown in Figure 2, the surface-milling operation can be identified in the time interval between approximately 18 and 80 seconds, excluding the instantaneous power peaks that do not correspond to steady cutting conditions. In Figure 3, the overall profile P_{tot} during the surfacing phase is shown.

To evaluate the power consumption per revolution, the mean, minimum, and maximum power values were computed for each spindle revolution, as reported in Table 1 and Figure 4.

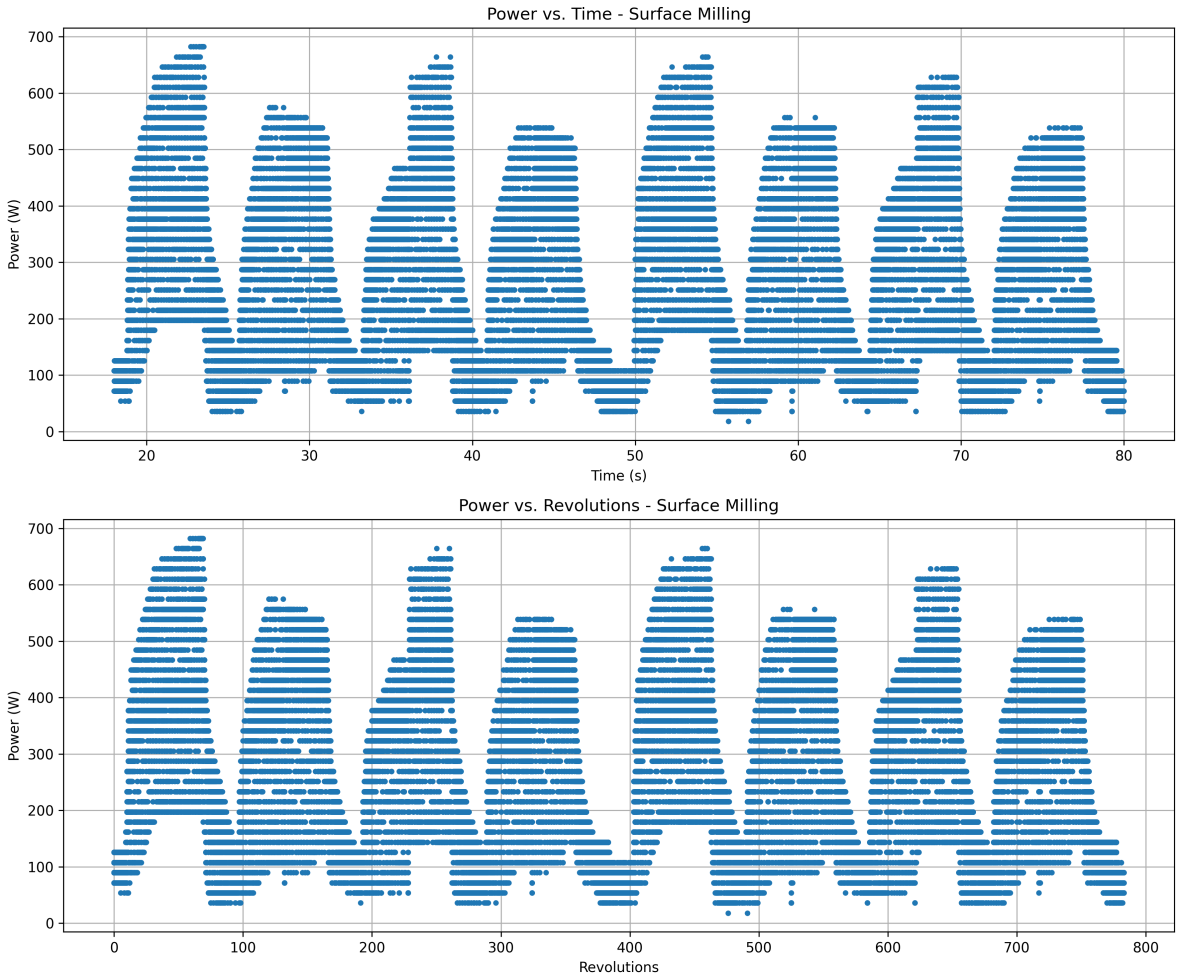


Figure 3: Overall power profile P_{tot} during the surfacing phase.

Table 1: Power consumption statistics.

Revolution	mean Power (W)	min Power (W)	max Power (W)
0	97.84	71.81	125.67
1	96.49	71.81	125.67
2	94.36	71.81	125.67
3	98.29	71.81	125.67
4	94.82	71.81	125.67
5	93.8	53.86	125.67
6	95.15	71.81	125.67
7	92.98	71.81	125.67
8	92.9	53.86	125.67
9	101.27	71.81	161.57
...	...	...	...

Energy Consumption per Revolution

Finally, the energy consumption for each revolution was evaluated. To compute the energy consumed during the i-th revolution, the following expression is used:

$$E_i = \int_{t_{0_i}}^{t_{f_i}} P(t)\, dt.$$

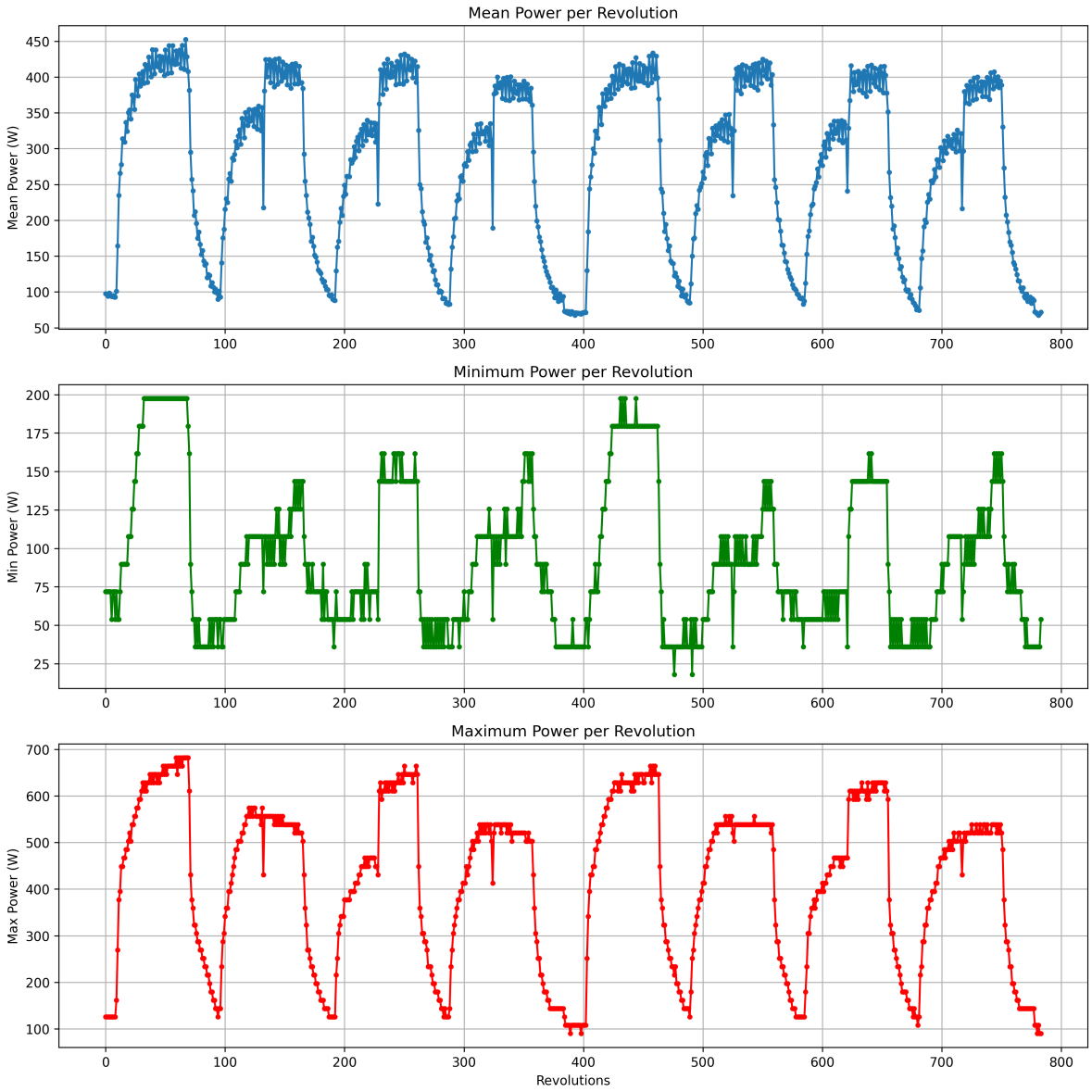


Figure 4: Power statistics profile.

Since the data are available as discrete samples, the trapezoidal rule is applied:

$$E_i = \sum_{j=i}^{i+n-1} \frac{P_j + P_{j+1}}{2} (t_{j+1} - t_j),$$

where n denotes the number of samples contained in the i-th revolution.

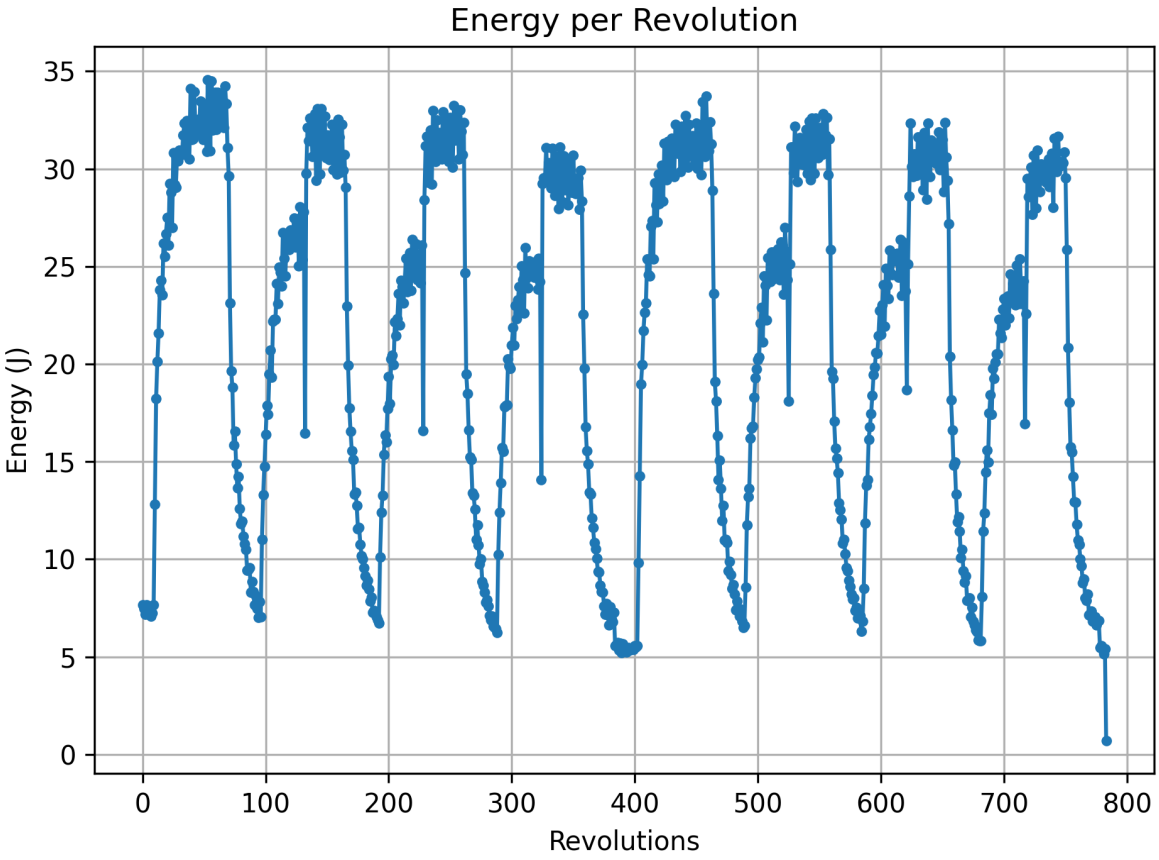


Figure 5: Energy profile.

Part 1 Smart Machining – Prediction and Monitoring

Chloé FUCHS

Report on second assignment



Exploring data from turning experiments

Anna-Carla ARAUJO

2025 – 2026

INTRODUCTION DRILLING ANALYSIS (CHLOE) 3

I. DATA COLLECTION 3

1. Data from INSA Students 3

1. Spindle power 3

2. Spindle Power Profile 4

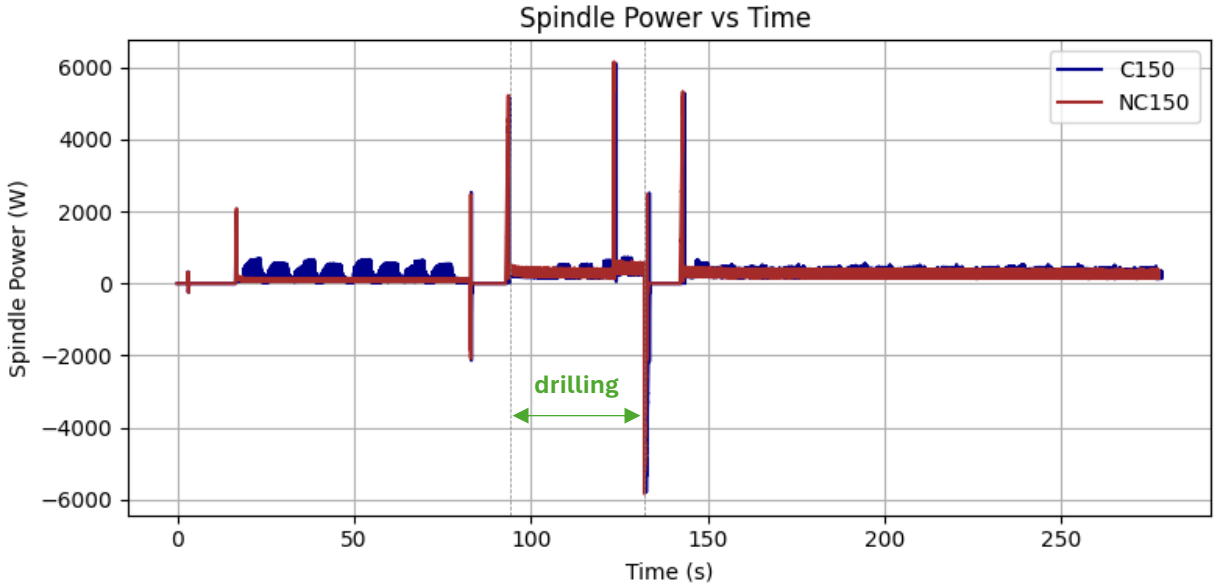
3. Energy per revolution 5

II. ANNEXES 5

Introduction Drilling analysis (Chloe)

In this study, the drilling operation was analyzed using experimental data provided by INSA students. The dataset includes synchronized measurements of spindle power under cutting and non-cutting conditions, recorded at a cutting speed of 150 m/min with a 5.5 mm diameter drill. Minimum, maximum, and average spindle power values were calculated, along with the energy consumed per revolution. This approach allows us to characterize the first drilling cycle.

The objective of this analysis is to establish a clear reference for the drilling operations, which can then be compared to other operations.



I. Data Collection

1. Data from INSA Students

- Cutting Speed V_C = 150 m/min (we assume it is the same as the pocket)
- Drill diameter D = 5.5 mm
- Files : 'Session5A1A.csv', 'Session5A1B.csv', 'TP1 GRC.mpf'
- Start time = 94s, End Time = 132s (graphic)

1. Spindle power

We know the power during cutting and non-cutting machining. To isolate the cutting power, we subtract the spindle power during non-cutting machining from the spindle power during cutting one.

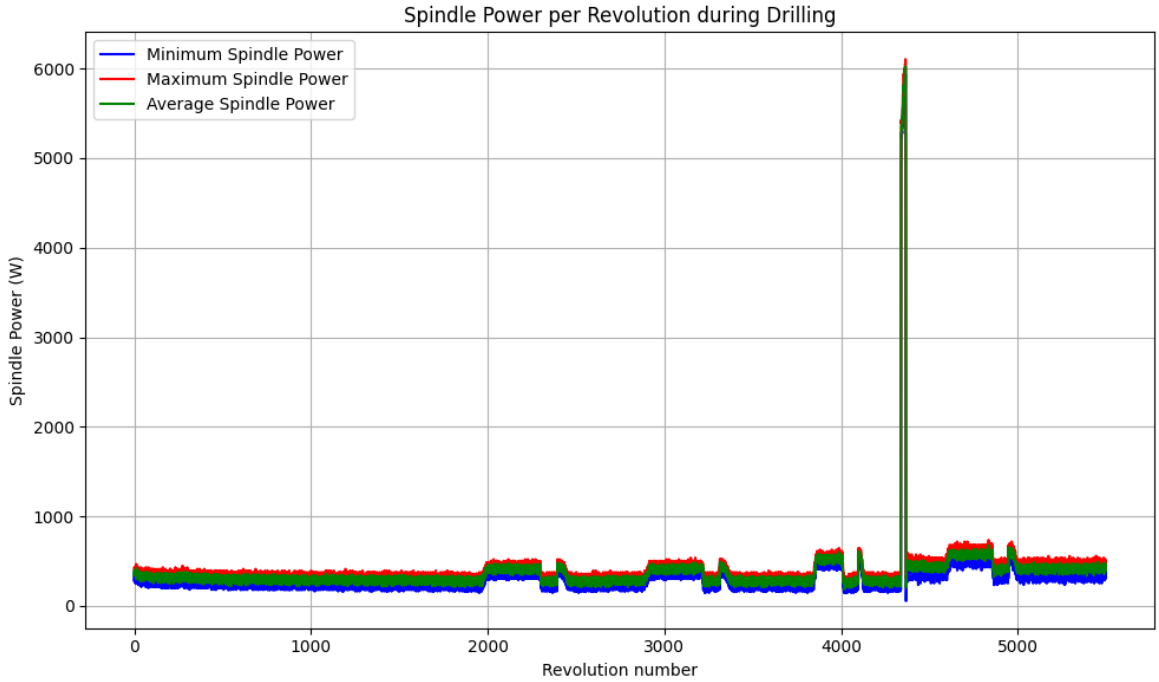
$$Pc = Spindle\ power\ during\ cutting - Spindle\ power\ during\ noncutting$$

But synchronizing of our tables, we still observe negative power values, which is impossible. So from now on consider the Spindle Power during Cutting instead of the Cutting one.

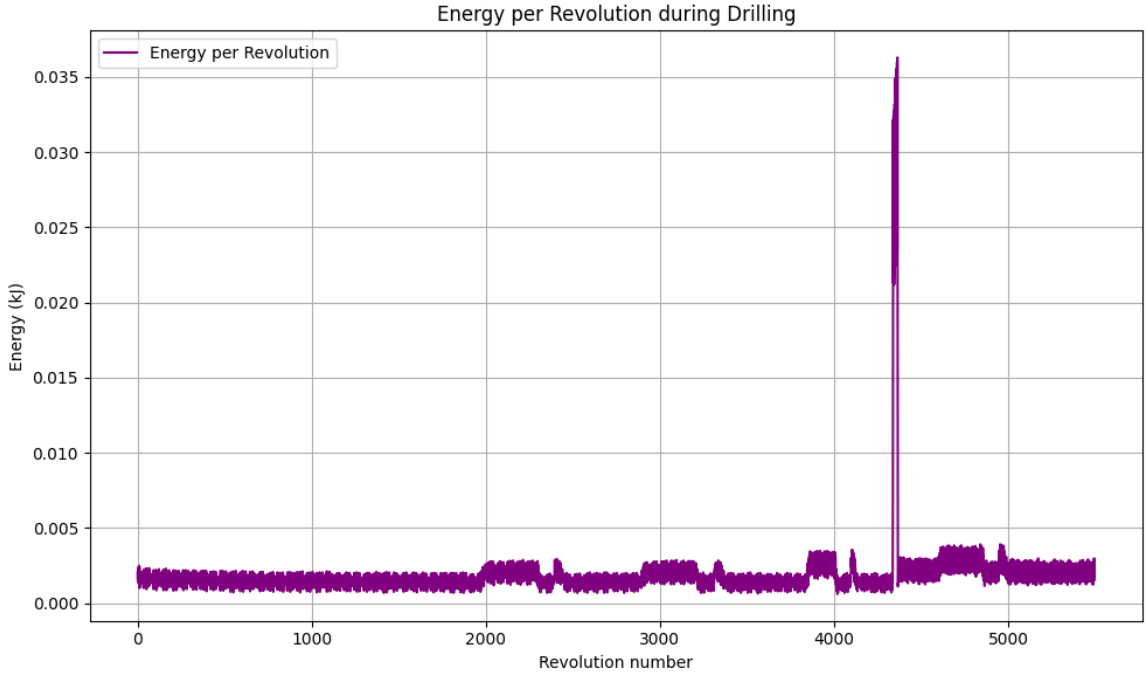
Table 1 : Spindle power (W)

revolution	min	max	avg
0	287.2	394.9	341.1
1	377.0	430.9	400.9
2	269.3	412.9	345.6
3	341.1	430.9	383.0
4	287.2	430.9	359.0
5	305.2	394.9	365.0
6	269.3	430.9	368.0
7	251.3	394.9	329.1
8	251.3	430.9	372.5
9	269.3	394.9	335.1
10	359.0	430.9	394.9
11	251.3	394.9	299.2
12	377.0	466.8	418.9
…	…	…	…

2. Spindle Power Profile



3. Energy per revolution



II. Annexes

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.ticker as ticker


# File data CSV
filedataC150 = 'Session5A1B.csv'
filedataNC150 = 'Session5A1A.csv'

# Reading csv
dfC150 = pd.read_csv(filedataC150, skiprows=10)
dfNC150 = pd.read_csv(filedataNC150, skiprows=10)

# # Columns names / Same for all data
dfC150.columns = [
    'time',
    'prog_pos_x',
    'meas_pos_x',
    'power',
    'prog_pos_z',
    'meas_pos_z',
    'prog_pos_y',
    'meas_pos_y'
]
dfNC150.columns = [
    'time',
    'prog_pos_x',
    'meas_pos_x',
    'power',
    'prog_pos_z',
    'meas_pos_z',
    'prog_pos_y',
    'meas_pos_y'
]

cols_to_convert = dfC150.columns.difference(['time'])  # or specify manually

# # Conversion
# for col in df.columns:
for df in [dfC150, dfNC150]:
    for col in cols_to_convert:
        df[col] = pd.to_numeric(df[col], errors='coerce')


# # Suppression des lignes incomplètes
dfC150 = dfC150.dropna()
dfNC150 = dfNC150.dropna()
```

```
49
50  # ========= GRAPHE Power =========
51  plt.figure(figsize=(8, 4))
52  plt.plot(dfNC150['time'], dfNC150['power'], label='NC150',
      color='darkblue')
53  plt.plot(dfC150['time'], dfC150['power'], label='C150', color='brown')
54  asymptotes_x = [107, 111,113.5,117.5, 120, 123,125, 129]
55  plt.axvline(x=94, color='gray', linestyle='--', linewidth=0.5)
56  plt.axvline(x=132, color='gray', linestyle='--', linewidth=0.5)
57  plt.xlabel('Time (s)')
58  plt.ylabel('Spindle Power (W)')
59  plt.title('Spindle Power vs Time ')
60  plt.grid(True)
61  plt.legend()
62  plt.tight_layout()
63  plt.show()
64
65  # Only keep rows where time exists in both cutting and non-cutting data
66  # Cutting Power for Vc=150
67  dfD150 = pd.merge(dfC150, dfNC150, on='time', suffixes=('_cut', '_
      nocut')) #we merge the two tabs by aligining their index
68  dfD150['power'] = dfD150['power_cut'] - dfD150['power_nocut']
69  # Keep only the useful columns :
70  dfD150 = dfD150[['time', 'power']]
71
72  #/!\ Pc < 0 impossible so we analyze the spindle power instead
73  dfD150 = dfC150[['time','power']]
74
75  # value
76  Vc = 150  # m/min
77  D = 5.5   # mm
78  n = (1000 * Vc) / (np.pi * D)  # rpm
79  T_rev = 60 / n  # durée d'une révolution en secondes
80
81  t0 = 94
82  tf = 132
83  f = 0.08
84  nb_revolutions = int(n*(tf-t0)/60)
85
86  # Liste des débuts de révolution
87  start_times = [t0 + i * T_rev for i in range(nb_revolutions)]
88  interval_dfsD150={}
89
90  print(f"Nombre de révolutions = {nb_revolutions}")
91  print("Premiers start_times :", start_times[:10])
92
93  #minimum and maximum Spindle Power
94  min_powersD150 = {}
95  max_powersD150 = {}
```

```
 96 avg_powersD150 = {}
 97
 98 results = []
 99 for i, start_time in enumerate(start_times, start=1):
100     end_time = start_time + T_rev
101     maskD = (dfD150['time'] >= start_time) & (dfD150['time'] < end_time)
102     interval_dfD150 = dfD150[maskD]
103
104     interval_dfsD150[f'intv{i}'] = interval_dfD150
105
106     min_powerD150 = interval_dfD150['power'].min()
107     max_powerD150 = interval_dfD150['power'].max()
108     avg_powerD150 = interval_dfD150['power'].mean()
109
110     min_powersD150[f'intv{i}'] = min_powerD150
111     max_powersD150[f'intv{i}'] = max_powerD150
112     avg_powersD150[f'intv{i}'] = avg_powerD150
113
114 rev=[i for i in range(nb_revolutions)]
115 # Build the table : Spindle Power
116 power_table = pd.DataFrame({
117     'revolution': rev[::1], #rev list
118     'min': list(min_powersD150.values()),
119     'max': list(max_powersD150.values()),
120     'avg': list(avg_powersD150.values()),
121 })
122
123 # Plot the three curves
124 plt.figure(figsize=(10,6))
125 plt.plot(power_table['revolution'], power_table['min'], label='Minimum
      Spindle Power', color='blue')
126 plt.plot(power_table['revolution'], power_table['max'], label='Maximum
      Spindle Power', color='red')
127 plt.plot(power_table['revolution'], power_table['avg'], label='Average
      Spindle Power', color='green')
128
129 plt.xlabel('Revolution number')
130 plt.ylabel('Spindle Power (W)')
131 plt.title('Spindle Power per Revolution during Drilling')
132 plt.legend()
133 plt.grid(True)
134 plt.tight_layout()
135 plt.show()
136
137
138
139 energy_per_rev = []
140
141 for i, start_time in enumerate(start_times, start=1):
```

```
142     end_time = start_time + T_rev
143     mask = (dfD150['time'] >= start_time) & (dfD150['time'] < end_time)
144     interval = dfD150[mask]
145 
146     if interval.empty:
147         energy_per_rev.append(np.nan)
148         continue
149 
150     # Compute delta_t inside the interval
151     dt = interval['time'].diff().fillna(0)
152     # Energy in Joules
153     energy = (interval['power'] * dt).sum()
154     # Convert to kJ
155     energy_kJ = energy / 1000.0
156 
157     energy_per_rev.append(energy_kJ)
158 
159 # Add to your DataFrame
160 power_table['energy_kJ'] = energy_per_rev
161 
162 # Plot energy curve
163 plt.figure(figsize=(10,6))
164 plt.plot(power_table['revolution'], power_table['energy_kJ'],
      label='Energy per Revolution', color='purple')
165 plt.xlabel('Revolution number')
166 plt.ylabel('Energy (kJ)')
167 plt.title('Energy per Revolution during Drilling')
168 plt.legend()
169 plt.grid(True)
170 plt.tight_layout()
171 plt.show()
172 
173 # Optional: Set revolution as index
174 power_table.set_index('revolution', inplace=True)
175 
176 # Export to CSV
177 power_table.to_csv('powers_summary.csv')
178 
```

SecondAssignment

November 15, 2025

1 Course of Multiagent Systems for smart machining: Second Assignment

Student: Daniele Trisotto

Email: daniele.trisotto@studenti.unitn.it or daniele.trisotto@eciu.eu

University of Trento

1.1 Assignment:

- Open data from last week:
  - Each student will analyze one operation
  - Compare power of operations:
  - Average per revolution:
  - Min/Max per revolution:
  - Profile of the power curve per revolution(during cutting)
  - Energy in one revolution:
    - Energy is the integral of the power.

1.2 Brief Summary of Data Processing Steps

My task is to analyze the pocket formation step so this notebook is focused on it.

1.3 Importing Data

[34]:
```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.ticker as ticker

# File data CSV
filedataC = 'Dati/Session5A1B.csv'                    #cut
filedataNC= 'Dati/Session5A1A.csv'                    #nocut

# Reading csv
dfC = pd.read_csv(filedataC, skiprows=10)
#dfC = dfC.iloc[:, :-1]
dfC = dfC.apply(pd.to_numeric, errors='coerce')
```

```
dfC = dfC.dropna()
dfNC = pd.read_csv(filedataNC, skiprows=10)
#dfNC = dfNC.iloc[:, :-1]
dfNC = dfNC.apply(pd.to_numeric, errors='coerce')
dfNC = dfNC.dropna()
```

[35]:
```
dfC.columns = ['time',
               'prog_pos_x',
               'meas_pos_x',
               'power',
               'prog_pos_y',
               'meas_pos_y',
               'prog_pos_z',
               'meas_pos_z']

dfNC.columns = ['time',
               'prog_pos_x',
               'meas_pos_x',
               'power',
               'prog_pos_y',
               'meas_pos_y',
               'prog_pos_z',
               'meas_pos_z']
```

1.4 Power visualization

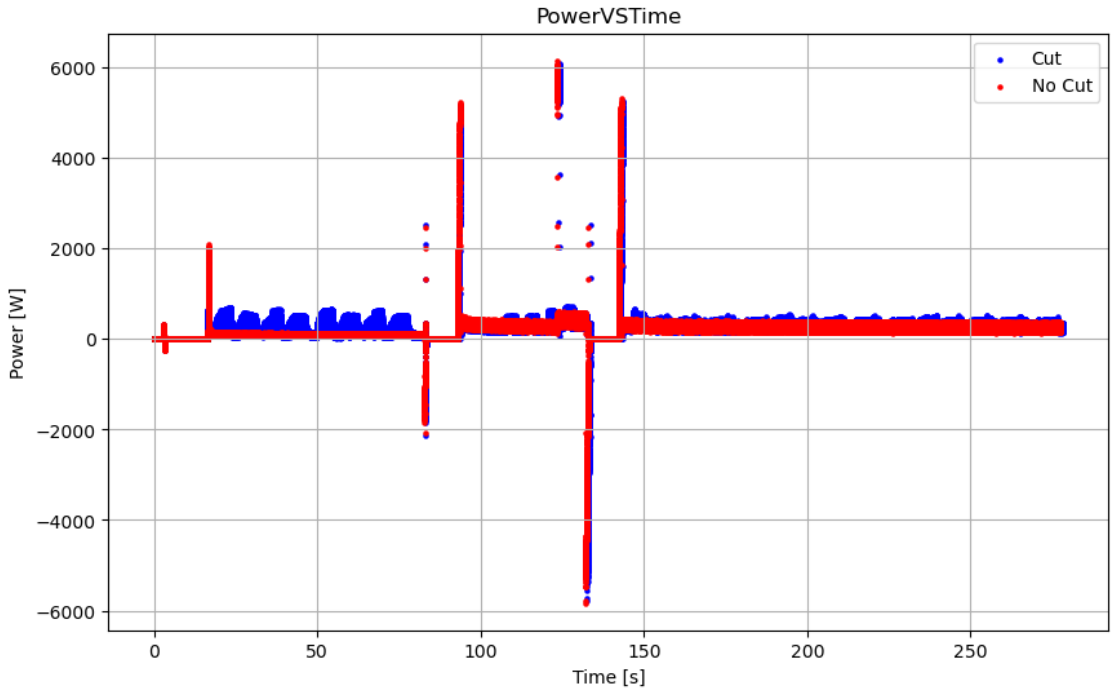
[36]:
```
plt.figure(figsize=(10,6))

# dfC
plt.scatter(dfC['time'], dfC['power'], label='Cut', color='blue', s=5) # s=20␣
 ↪dimensione punti

# dfNC
plt.scatter(dfNC['time'], dfNC['power'], label='No Cut', color='red', s=5)

# Labels
plt.xlabel('Time [s]')
plt.ylabel('Power [W]')
plt.title('PowerVSTime')
plt.legend()
plt.grid(True)

plt.show()
```



1.5 Position Visualization of the full process:

[37]:
```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Scegli il DataFrame
df = dfC  # oppure dfNC

# Crea figura e asse 3D
fig = plt.figure(figsize=(10,7))
ax = fig.add_subplot(111, projection='3d')

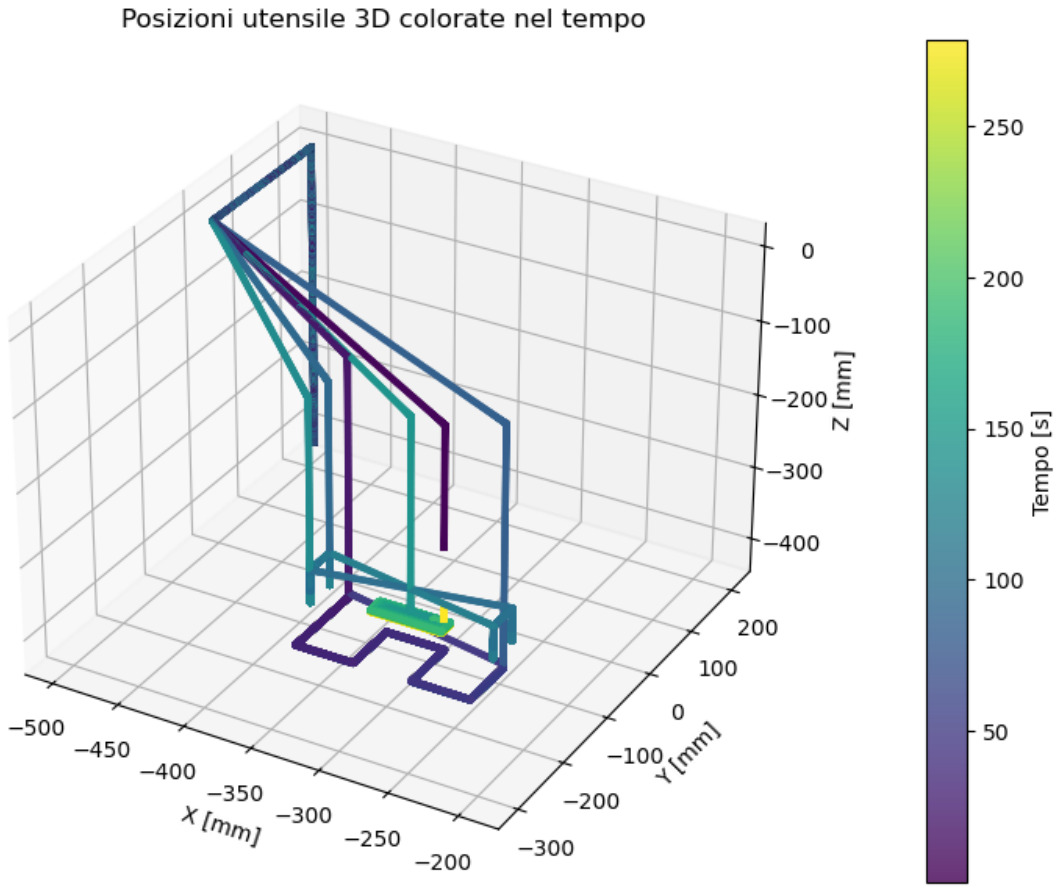
# Scatter 3D con colore direttamente dal tempo
sc = ax.scatter(df['meas_pos_x'], df['meas_pos_y'], df['meas_pos_z'],
                c=df['time'], cmap='viridis', s=5, alpha=0.8)

# Etichette assi
ax.set_xlabel('X [mm]')
ax.set_ylabel('Y [mm]')
ax.set_zlabel('Z [mm]')
ax.set_title('Posizioni utensile 3D colorate nel tempo')

# Barra colore per il tempo
cbar = plt.colorbar(sc, ax=ax, pad=0.1)
```

```
cbar.set_label('Tempo [s]')

plt.show()
```



[38]:
```
# choose DataFrame
df = dfC  # oppure dfNC

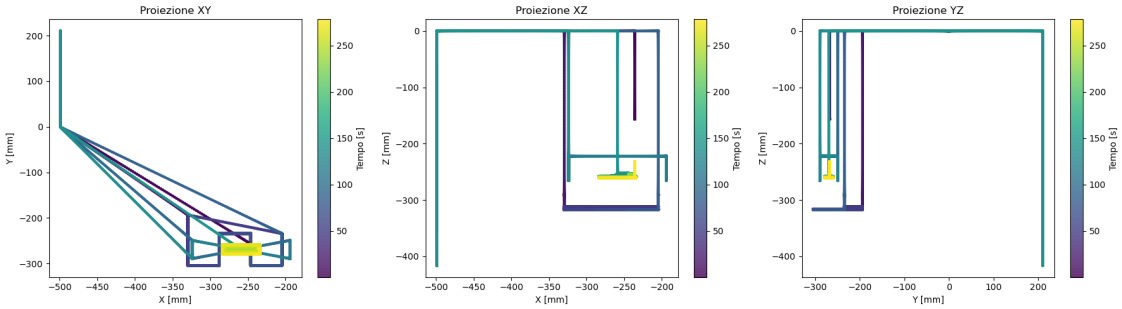
# Imposta figure e dimensioni
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 1 Proiezione XY
sc1 = axes[0].scatter(df['meas_pos_x'], df['meas_pos_y'], c=df['time'],␣
 ↪cmap='viridis', s=5, alpha=0.8)
axes[0].set_xlabel('X [mm]')
axes[0].set_ylabel('Y [mm]')
axes[0].set_title('Proiezione XY')
cbar1 = plt.colorbar(sc1, ax=axes[0])
cbar1.set_label('Tempo [s]')
```

```
# 2 Proiezione XZ
sc2 = axes[1].scatter(df['meas_pos_x'], df['meas_pos_z'], c=df['time'],␣
 ↪cmap='viridis', s=5, alpha=0.8)
axes[1].set_xlabel('X [mm]')
axes[1].set_ylabel('Z [mm]')
axes[1].set_title('Proiezione XZ')
cbar2 = plt.colorbar(sc2, ax=axes[1])
cbar2.set_label('Tempo [s]')

# 3 Proiezione YZ
sc3 = axes[2].scatter(df['meas_pos_y'], df['meas_pos_z'], c=df['time'],␣
 ↪cmap='viridis', s=5, alpha=0.8)
axes[2].set_xlabel('Y [mm]')
axes[2].set_ylabel('Z [mm]')
axes[2].set_title('Proiezione YZ')
cbar3 = plt.colorbar(sc3, ax=axes[2])
cbar3.set_label('Tempo [s]')

plt.tight_layout()
plt.show()
```



From this plot it is clear that the pocket formation is the last operation

1.6 Position Visualization of the Pocket formation

[39]:
```
# Scegli DataFrame
df = dfC  # oppure dfNC

# Soglia temporale
t_soglia = 144.7 # ad esempio 50 secondi
t_soglia2 = 278.1
df_filtered = df[(df['time'] > t_soglia) & (df['time'] < t_soglia2) ]

# Imposta figure e dimensioni
```

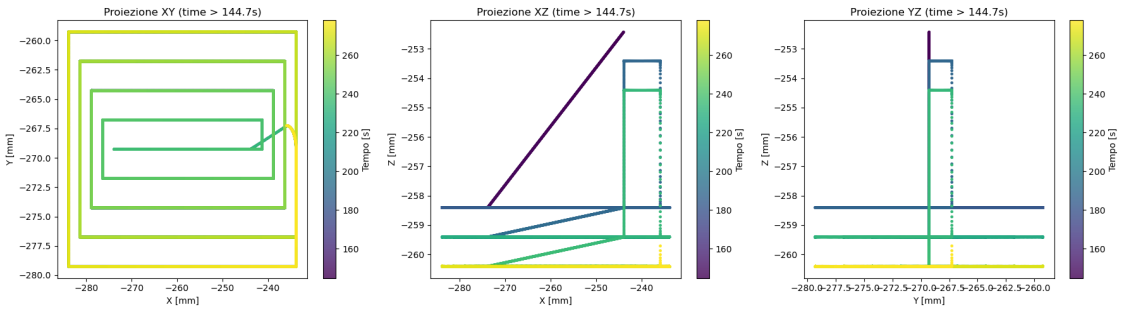
```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 1 Proiezione XY
sc1 = axes[0].scatter(df_filtered['meas_pos_x'], df_filtered['meas_pos_y'],
                      c=df_filtered['time'], cmap='viridis', s=5, alpha=0.8)
axes[0].set_xlabel('X [mm]')
axes[0].set_ylabel('Y [mm]')
axes[0].set_title(f'Proiezione XY (time > {t_soglia}s)')
cbar1 = plt.colorbar(sc1, ax=axes[0])
cbar1.set_label('Tempo [s]')

# 2 Proiezione XZ
sc2 = axes[1].scatter(df_filtered['meas_pos_x'], df_filtered['meas_pos_z'],
                      c=df_filtered['time'], cmap='viridis', s=5, alpha=0.8)
axes[1].set_xlabel('X [mm]')
axes[1].set_ylabel('Z [mm]')
axes[1].set_title(f'Proiezione XZ (time > {t_soglia}s)')
cbar2 = plt.colorbar(sc2, ax=axes[1])
cbar2.set_label('Tempo [s]')

# 3 Proiezione YZ
sc3 = axes[2].scatter(df_filtered['meas_pos_y'], df_filtered['meas_pos_z'],
                      c=df_filtered['time'], cmap='viridis', s=5, alpha=0.8)
axes[2].set_xlabel('Y [mm]')
axes[2].set_ylabel('Z [mm]')
axes[2].set_title(f'Proiezione YZ (time > {t_soglia}s)')
cbar3 = plt.colorbar(sc3, ax=axes[2])
cbar3.set_label('Tempo [s]')

plt.tight_layout()
plt.show()
```



[40]:
```
# Scegli DataFrame
df = dfC  # oppure dfNC
```

```
# Soglia temporale
t_soglia = 144.7 # ad esempio 50 secondi
t_soglia2 = 278.1
df = df[(df['time'] > t_soglia) & (df['time'] < t_soglia2) ]

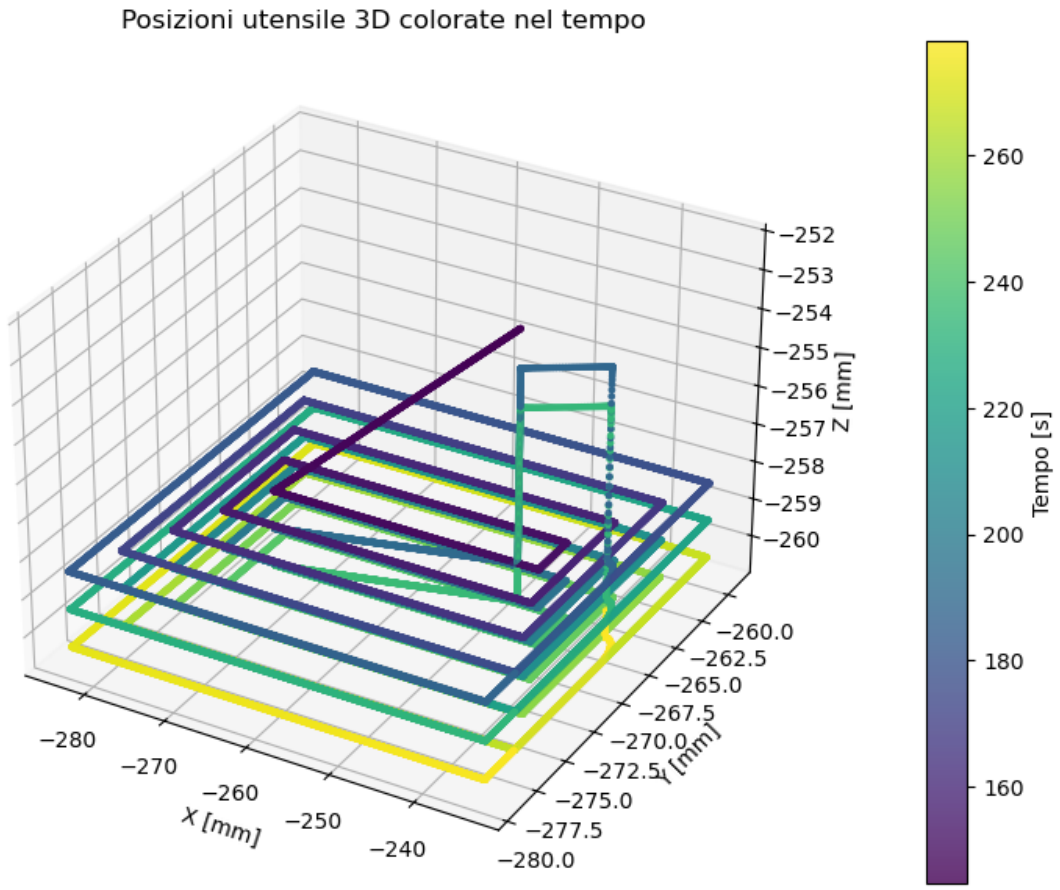
# Crea figura e asse 3D
fig = plt.figure(figsize=(10,7))
ax = fig.add_subplot(111, projection='3d')

# Scatter 3D con colore direttamente dal tempo
sc = ax.scatter(df['meas_pos_x'], df['meas_pos_y'], df['meas_pos_z'],
                c=df['time'], cmap='viridis', s=5, alpha=0.8)

# Etichette assi
ax.set_xlabel('X [mm]')
ax.set_ylabel('Y [mm]')
ax.set_zlabel('Z [mm]')
ax.set_title('Posizioni utensile 3D colorate nel tempo')

# Barra colore per il tempo
cbar = plt.colorbar(sc, ax=ax, pad=0.1)
cbar.set_label('Tempo [s]')

plt.show()
```



1.7 Power of the pocket formation task:

[41]:
```
plt.figure(figsize=(10,6))

# dfC
plt.scatter(dfC['time'], dfC['power'], label='Cut', color='blue', s=5)

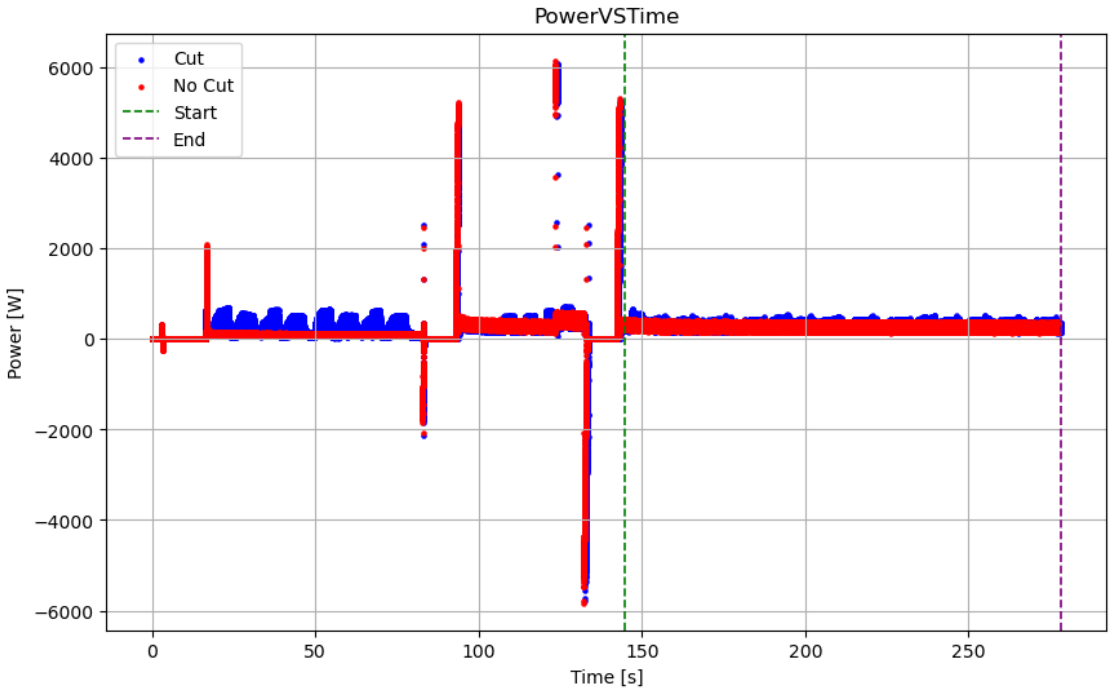
# dfNC
plt.scatter(dfNC['time'], dfNC['power'], label='No Cut', color='red', s=5)

# Linee verticali
plt.axvline(x=144.7, color='green', linestyle='--', linewidth=1.2,␣
 ↪label='Start')
plt.axvline(x=278.1, color='purple', linestyle='--', linewidth=1.2, label='End')

# Labels
plt.xlabel('Time [s]')
plt.ylabel('Power [W]')
plt.title('PowerVSTime')
```

```
plt.legend()
plt.grid(True)

plt.show()
```



[42]:
```
t_min = 144.7
t_max = 278.1

# Filtra dfC
dfC = dfC[(dfC['time'] >= t_min) & (dfC['time'] <= t_max)].copy()

# Filtra dfNC
dfNC = dfNC[(dfNC['time'] >= t_min) & (dfNC['time'] <= t_max)].copy()
```

1.8 Power difference:

[43]:
```
# Unisci tramite merge_asof (accoppiamento per tempo più vicino)
df_merged = pd.merge_asof(
    dfC.sort_values('time'),
    dfNC.sort_values('time'),
    on='time',
    suffixes=('_C', '_NC')
)
```

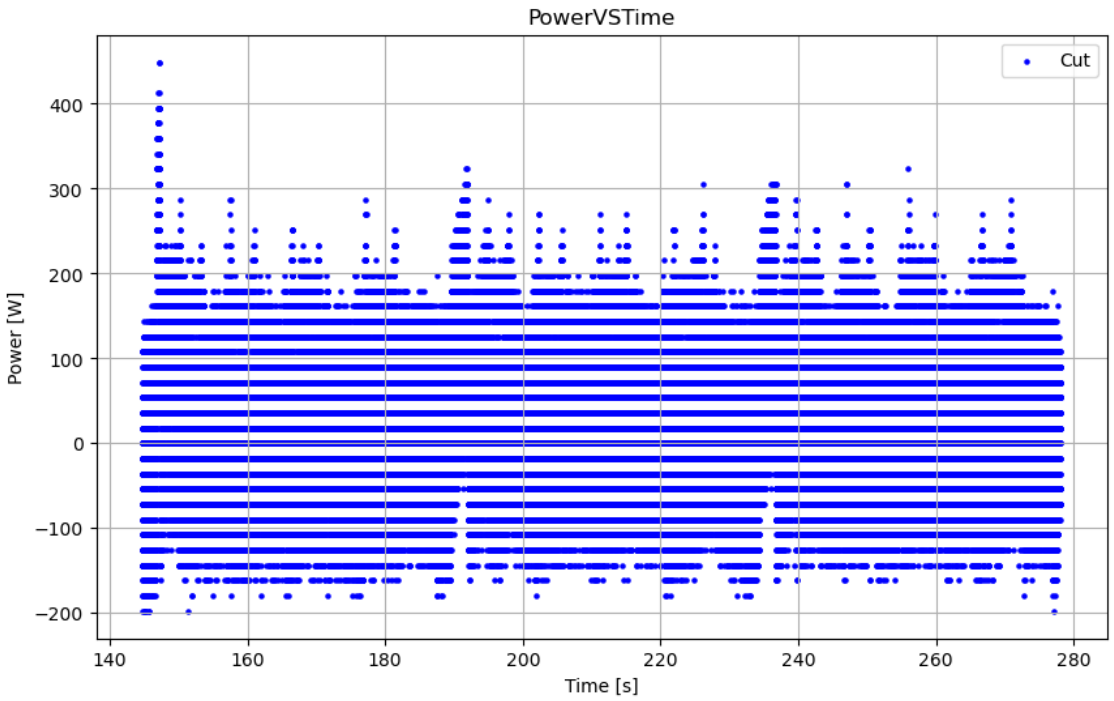
```
# Calcola differenza
df_diff = pd.DataFrame({
    'time': df_merged['time'],
    'power_diff': df_merged['power_C'] - df_merged['power_NC']
})

plt.figure(figsize=(10,6))

plt.scatter(df_diff['time'], df_diff['power_diff'], label='Cut', color='blue',␣
 ↪s=5)

plt.xlabel('Time [s]')
plt.ylabel('Power [W]')
plt.title('PowerVSTime')
plt.legend()
plt.grid(True)

plt.show()
```



From the previous plot we notice that often the cutting power is negative. This is not possible and probably comes out from a data inconsistency. So from now on we consider only total power not cutting power

1.9 Machining Parameters:

Here I report the full programming code:

NOTE: I have never learned about CNC programming and in any video of this course isn't explained CNC programming su I interpreted commands through an IA.

```
E_HEAD(272113663,0,0,0,170,-70,-15,71,17,5,25,0,1,20,,4,7,6,100,1,0,33279);*RO*
$AN_SLTRACE=1 E_MI_PL("FR_SURF_D63_ALU","FORET5.5_CARB_FSI",1,0.1,2,400,2,13,0,90,-1,90,0.5,0,
E_DR_PEC(1,0,0,"FORET5.5_CARB_FSI","FR2T D10 ALU FSI",1,0.08,3,80,2,40,-1.5,90,1.5,90,0,0.6,1,
_E_P001: E_PS_SEQ(1,0,0,0,90,20,90,-15,90,20,90,-55,90,150,90,-15,90,150,90,-55,90,,0,,0,,0,,0
E_DR_PEC(1,0,0,"FORET5.5_CARB_FSI","",1,0.08,3,80,2,40,-18,90,14,90,4,0.6,1,1.4,1.6,90);*RO*
_E_P002: E_PS_SEQ(1,0,0,0,90,,0,,0,,0,,0,150,90,-15,90,150,90,-55,90,,0,,0,,0,,0
```

[44]: <pandas.io.formats.style.Styler at 0x272d0247a10>

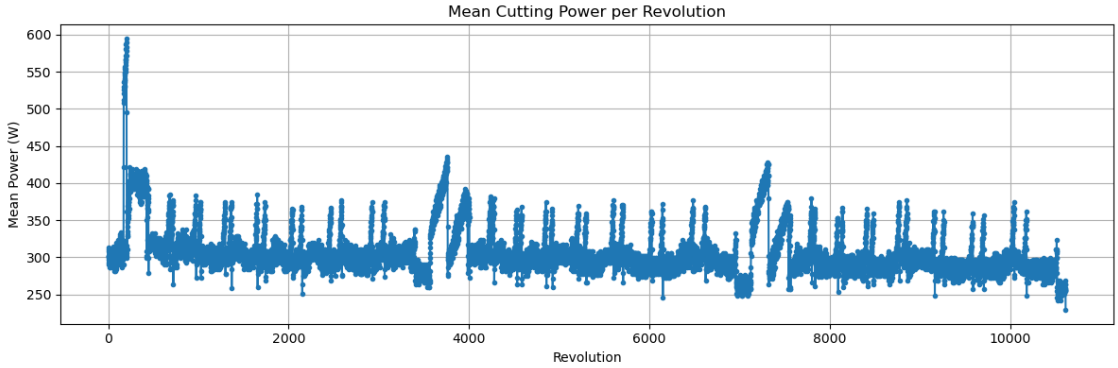
[45]:
```
fig, axes = plt.subplots(3, 1, figsize=(12, 12))
fig.subplots_adjust(hspace=0.4)

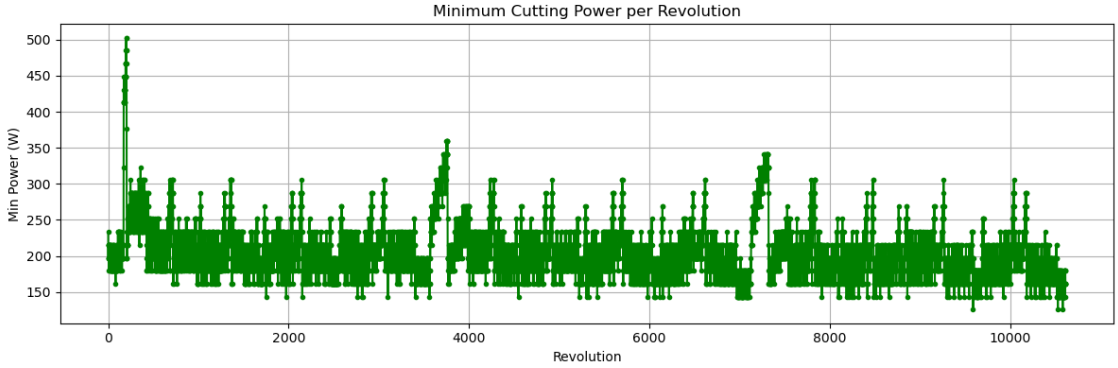
axes[0].plot(stats['rev'], stats['mean_power'], marker='.')
axes[0].set_xlabel('Revolution')
axes[0].set_ylabel('Mean Power (W)')
axes[0].set_title('Mean Cutting Power per Revolution')
axes[0].grid(True)

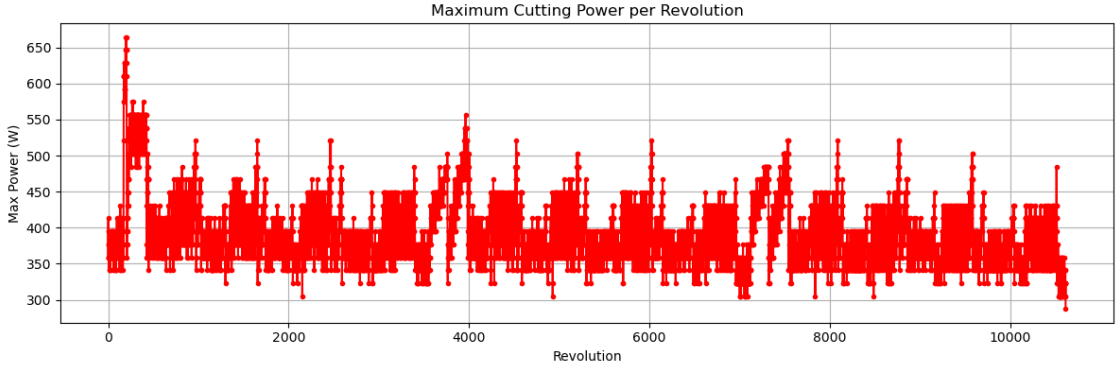
axes[1].plot(stats['rev'], stats['min_power'], marker='.', color='green')
axes[1].set_xlabel('Revolution')
axes[1].set_ylabel('Min Power (W)')
axes[1].set_title('Minimum Cutting Power per Revolution')
axes[1].grid(True)

axes[2].plot(stats['rev'], stats['max_power'], marker='.', color='red')
axes[2].set_xlabel('Revolution')
axes[2].set_ylabel('Max Power (W)')
axes[2].set_title('Maximum Cutting Power per Revolution')
axes[2].grid(True)

plt.tight_layout()
plt.show()
```







[46]:
```
energy_per_rev = []

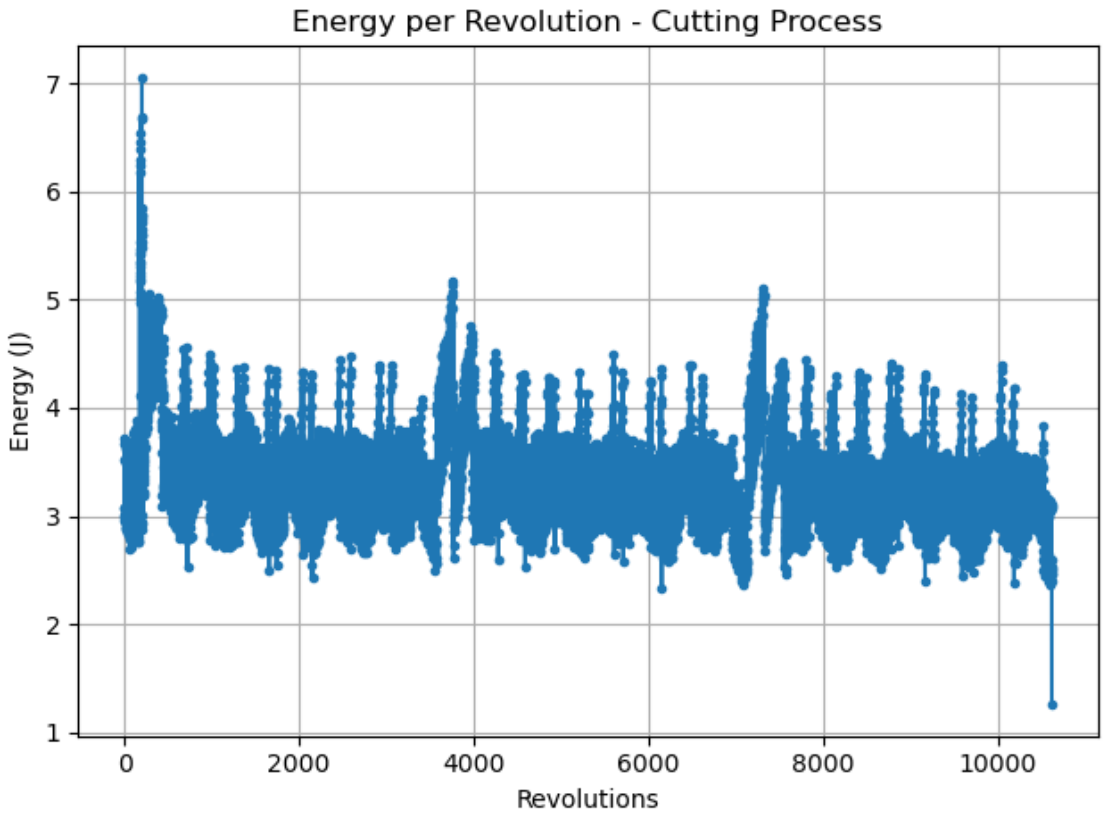
for rev_id, group in dfC.groupby('rev'):
    t = group['time'].values
    p = group['power'].values

    dt = t[1:] - t[:-1]                  # Δt_j
    p_avg = (p[1:] + p[:-1]) / 2         # (P_j + P_{j+1})/2b

    E = (p_avg * dt).sum()

    energy_per_rev.append([rev_id, E])
```

```
energy_per_rev = pd.DataFrame(energy_per_rev, columns=['rev', 'energy'])
plt.plot(energy_per_rev['rev'], energy_per_rev['energy'], marker='.',␣
 ↪linestyle='-')
plt.xlabel('Revolutions')
plt.ylabel('Energy (J)')
plt.title('Energy per Revolution - Cutting Process')
plt.grid(True)
plt.tight_layout()
plt.show()
```



Comparison between the different operations

When comparing the three machining operations, both the power and energy profiles clearly reflect the underlying process kinematics. Despite the different spindle speeds imposed by the varying tool diameters, the instantaneous power traces consistently reveal the structure of each operation. In drilling, the step-like pattern corresponds directly to the sequence of four holes, with each plateau representing one drilling event. Conversely, in surface milling and pocket formation, the oscillatory behaviour of the power signal is associated with the continuous changes in tool engagement caused by the toolpath direction reversals on the workplane.

The average power per revolution falls within a similar range for all operations—approximately 200 to 500 W/rev. This is coherent with the fact that the cutting speed is kept constant across the three processes and the work material does not change.

Regarding energy, the profiles exhibit the same underlying behaviour observed in the power signals, since energy is obtained by integrating power over time. When analysing the per-revolution values, surface milling shows the highest energy consumption, with approximately 30 J per revolution (or slightly below), whereas drilling and pocket formation remain around 4 J per revolution.